# Code Bharat Compiler Interview Guide

Generated from this repository on 2026-03-24 14:28

**Goal:** explain how the compiler service works, how code moves from request to execution, how temporary files are created and cleaned, and how the backend depends on this service.

## Quick Interview Memory Map

| Topic | What you should say |
|---|---|
| **JavaScript** | This project uses JavaScript syntax to define routes, Promises, async/await, callbacks, imports, and shell command strings. |
| **Node.js** | Node.js is the server runtime that runs the JavaScript and exposes server-side modules like fs, path, process.env, and child_process. |
| **Express** | Express is used to define HTTP endpoints such as /run and /ai-review, parse JSON, and send responses. |
| **Compiler flow** | Request comes in -> temp files are created -> language-specific executor runs -> output is collected -> temp files are cleaned. |
| **Strong answer** | It is a microservice design. The main backend delegates execution work to a separate compiler service, which keeps the judge backend cleaner. |

## JavaScript vs Node.js in This Project

JavaScript is the language used to write the service logic. Node.js is the runtime that executes that JavaScript on the server and gives access to operating-system features.

In this project, JavaScript handles route logic, validation, Promises, async flow, and command construction. Node.js gives the code access to files, paths, environment variables, and child processes.

### JavaScript language features used here

- CommonJS modules with require(...) and module.exports.
- Object destructuring with default values in req.body handling.
- Arrow functions for route handlers and callbacks.
- async/await for readable asynchronous flow.
- Promises to wrap child_process.exec and make executors awaitable.
- Template literals to build shell commands with file paths.
- try/catch for converting failures into JSON responses.

### Node.js runtime modules used here

- `express` for the HTTP server and routes.
- `cors` for cross-origin access.
- `fs` for writing and deleting temp files.
- `path` for directory-safe path creation.

- `child_process.exec` for running compilers and interpreters.
- `dotenv` for loading .env values into process.env.
- `uuid` for unique temp file names.

### How to explain the /run route in interview

1. Client sends language, code, and optional input as JSON.
2. Server validates that code is present.
3. Service creates a temp source file and a temp input file.
4. Service picks the correct executor for the selected language.
5. Executor compiles and runs, or directly interprets, with stdin redirection.
6. stdout is returned on success and stderr/error details are returned on failure.
7. Cleanup removes temporary files after execution.

### Good interview answers you can say directly

> **Q: Why Node.js for this service?**
> A: Because the service mainly orchestrates requests, files, child processes, and JSON responses. Node.js is lightweight and fits that pattern well.
>
> **Q: What is JavaScript doing here?**
> A: JavaScript defines the service logic: routes, validation, async flow, command construction, and error handling.
>
> **Q: What is Node.js doing here?**
> A: Node.js provides the runtime and built-in APIs that let the JavaScript interact with the OS, filesystem, processes, and environment variables.
>
> **Q: Why separate the compiler service from the main backend?**
> A: It keeps execution isolated as its own service, which makes the architecture more modular and easier to debug or deploy independently.
>
> **Q: What limitations would you mention honestly?**
> A: There is no timeout, queue, sandbox, or strict resource isolation yet, so this is good for a project/demo system but would need stronger controls in production.

### 1. Compiler Service Overview

This compiler is a separate Express microservice that receives code, writes temporary source and input files, runs language-specific compile/execute commands, returns output, and optionally asks Gemini for AI review.

### 2. Run Request Flow

8. Frontend or backend sends `POST /run` with `language`, `code`, and optional `input`.
9. Service validates that code is not empty.
10. Service writes source code to `codes/` and input text to `inputs/`.
11. Service chooses executor by language.
12. Executor shells out to native toolchain/interpreter with stdin redirection.
13. Executor resolves stdout or rejects with error details.
14. Cleanup helper removes temporary files.

15. Service returns output JSON to caller.

### 3. AI Review Flow

16. Frontend sends `POST /ai-review` with `code`.
17. Service validates code presence.
18. Service calls Gemini through `aiCodeReview.js`.
19. Generated review text is returned to the frontend.

### 4. Source Tree

```
Compiler/
 - .dockerignore
 - .env
 - .gitignore
 - Dockerfile
 - Index.js
 - aiCodeReview.js
 - executeC.js
 - executeCpp.js
 - executeGo.js
 - executeJava.js
 - executeJs.js
 - executePython.js
 - generateFile.js
 - generateInputFile.js
 - package.json
   - cleanup.js
```

### 5. Environment Variables

| Variable | Use |
|---|---|
| PORT | Compiler service listen port |
| GOOGLE_GEMINI_API_KEY | Gemini API key used by `/ai-review` |

### 6. Key Interview Observations

- This service is execution-orchestration heavy and depends on OS-level compilers/interpreters, so deployment environment matters.
- The backend submission flow depends on this service for every hidden test case.
- Java executor exists but `/run` does not currently dispatch to it.
- Fixed `Main.java` naming can cause concurrency issues for parallel Java jobs.
- There is no timeout, sandboxing, queueing, or resource isolation beyond basic process execution.
- Temp-file cleanup is present, which is good, but failures are only logged.

### Redacted Compiler `.env`

```
1: PORT=***REDACTED***
2: GOOGLE_GEMINI_API_KEY=***REDACTED***
```

### Compiler Package Manifest

**File:** Compiler/package.json

**Purpose:** Defines the compiler service runtime, scripts, and dependencies.

**How it fits into service flow:** This file shapes how the compiler service starts and which libraries are available to `/run` and `/ai-review`.

**Interview-ready talking points:**

- The service is a Node CommonJS app started from `Index.js`.
- Dependencies are intentionally small: Express, CORS, dotenv, UUID, and Google GenAI.

**Implementation notes:**

- Package name is still `backend`, which is misleading for a separate compiler service.

### Code

```
 1: {
 2:   "name": "backend",
 3:   "version": "1.0.0",
 4:   "description": "",
 5:   "main": "index.js",
 6:   "engines": {
 7:     "node": "20.x"
 8:   },
 9:   "scripts": {
10:     "dev": "node Index.js",
11:     "start": "node Index.js",
12:     "test": "echo \"Error: no test specified\" && exit 1"
13:   },
14:   "keywords": [],
15:   "author": "",
16:   "license": "ISC",
17:   "type": "commonjs",
18:   "dependencies": {
19:     "@google/genai": "^1.12.0",
20:     "cors": "^2.8.5",
21:     "dotenv": "^17.2.0",
22:     "express": "^5.1.0",
23:     "uuid": "^11.1.0"
24:   }
25: }

Walkthrough
```

Lines 9-16: Defines service start commands.

Line 17: Declares CommonJS module mode.

Lines 18-25: Lists runtime dependencies used by the compiler service.

### Compiler Git Ignore Rules

**File:** Compiler/.gitignore

**Purpose:** Prevents secrets, generated temp files, and local artifacts from being committed.

**How it fits into service flow:** Does not affect request handling but protects repo hygiene.

**Interview-ready talking points:**

- Generated source/input/output temp folders are ignored.
- Secrets and private keys are intended to be ignored.

**Implementation notes:**

- Ignore rule uses `keys/` while repo folder is `Keys/`; Windows is case-insensitive, but other environments may not be.

Code

```
 1: .env
 2: keys/
 3: *.pem
 4:
 5: node_modules/
 6: dist/
 7: dist-ssr/
 8:
 9: logs/
10: *.log
11: npm-debug.log*
12: yarn-debug.log*
13: yarn-error.log*
14: pnpm-debug.log*
15: lerna-debug.log*
16:
17: .vscode/
18: !.vscode/extensions.json
19: .idea/
20: .DS_Store
21: *.suo
22: *.ntvs*
23: *.njsproj
24: *.sln
25: *.sw?
26:
27: codes/
28: outputs/
29: inputs/

Walkthrough
```

Line 1: Ignores secret environment file.

Lines 2-4: Intends to ignore private key folder.

Lines 5-26: Ignores installed dependencies.

Line 27: Ignores generated source files.

Line 28: Ignores compiled binaries.

Line 29: Ignores generated input files.

### Compiler Docker Ignore Rules

**File:** Compiler/.dockerignore

**Purpose:** Keeps Docker build context small by excluding local and generated artifacts.

**How it fits into service flow:** Affects container build behavior only.

**Interview-ready talking points:**

- Generated temp folders are excluded from Docker context.

**Implementation notes:**

- Lockfile is ignored here.

### Code

```
1: node_modules
2: package-lock.json
3: codes
4: outputs
5: inputs

Walkthrough
```

Line 1: Skips local dependencies.

Line 2: Skips package lockfile in Docker context.

Line 3: Skips generated code files.

Line 4: Skips compiled binaries.

Line 5: Skips generated input files.

### Compiler Dockerfile

**File:** Compiler/Dockerfile

**Purpose:** Builds a runnable container with native toolchains for compiling and executing user code.

**How it fits into service flow:** Defines the environment that the compiler service needs in production: C/C++ build tools, Python, Go, and JDK.

**Interview-ready talking points:**

- Installs native toolchains because `/run` shells out to system compilers/interpreters.
- Exposes port 7000, matching the service default.

**Implementation notes:**

- Docker uses Node 18, but `package.json` engines says 20.x. That mismatch is worth discussing in interviews.

### Code

```
 1: FROM node:18
 2:
 3: WORKDIR /app
 4:
 5: RUN apt-get update && apt-get install -y \
 6:   build-essential \
 7:   python3 \
 8:   golang \
 9:   default-jdk \
10:   && apt-get clean \
11:   && rm -rf /var/lib/apt/lists/*
12:
13: COPY .env .env
14: COPY package*.json ./
15: RUN npm install
16:
17: COPY . .
18:
19: EXPOSE 7000
20:
21: CMD ["node", "Index.js"]

Walkthrough
```

Lines 1-4: Selects the base Node image.

Lines 5-12: Installs required native toolchains.

Line 13: Copies environment file into image.

Line 14: Copies package metadata before install.

Lines 15-16: Installs service dependencies.

Lines 17-18: Copies service source code.

Lines 19-20: Documents service port.

Line 21: Starts compiler service.

### Compiler Service Entrypoint

**File:** Compiler/Index.js

**Purpose:** Creates the Express service, exposes `/run` and `/ai-review`, writes temp files, dispatches execution by language, and returns results.

**How it fits into service flow:** Backend `submitSolution` and frontend `ProblemPage` both depend on this service for code execution and AI review.

**Interview-ready talking points:**

- This file is the orchestration layer for compiler operations.
- Run flow goes request -> validate code -> generate temp code/input files -> select executor -> return output.
- AI review flow goes request -> Gemini call -> markdown/text review response.

**Implementation notes:**

- `executeJava` is implemented in the repo but not wired into the `/run` language switch.
- Returned response includes file paths, which is useful for debugging but may expose internal temp-path details.

### Code

```
 1: const express = require('express');
 2: const app = express();
 3: const { generateFile } = require('./generateFile');
 4: const { generateInputFile } = require('./generateInputFile');
 5: const cors = require('cors');
 6: const { aiCodeReview } = require('./aiCodeReview.js');
 7: const { executeCpp } = require('./executeCpp');
 8: const { executePython } = require('./executePython');
 9: const { executeJs } = require('./executeJs');
10: const { executeGo } = require('./executeGo');
11: const { executeC } = require('./executeC.js');
12:
13: app.use(cors());
14: app.use(express.urlencoded({ extended: true }));
15: app.use(express.json());
16:
17: app.get("/", (req, res) => {
18:     res.json({ online: 'compiler' });
19: });
20:
21: app.post("/run", async (req, res) => {
22:     const { language = 'cpp', code, input = '' } = req.body;
23:
24:     if (code === undefined || code.trim() === '') {
25:         return res.status(400).json({
26:             success: false,
27:             error: "Empty code! Please provide some code to execute."
28:         });
29:     }
30:
31:     try {
32:         const filePath = await generateFile(language, code);
33:
34:         const inputPath = await generateInputFile(input);
35:
36:         let output;
37:
38:         if (language === "cpp") {
39:             output = await executeCpp(filePath, inputPath);
40:         } else if (language === "c") {
41:             output = await executeC(filePath, inputPath);
42:         } else if (language === "python") {
43:             output = await executePython(filePath, inputPath);
44:         } else if (language === "javascript") {
45:             output = await executeJs(filePath, inputPath);
46:         } else if (language === "go") {
47:             output = await executeGo(filePath, inputPath);
48:         } else {
49:             throw new Error("Unsupported language");
50:         }
51:
52:         res.json({
53:             success: true,
54:             filePath,
55:             inputPath,
56:             output
57:         });
58:     } catch (error) {
59:         console.error('Error executing code:', error);
```

```
60:
61:         res.status(500).json({
62:             success: false,
63:             error: error.message || error.toString() || 'An error occurred while executing the
code'
64:         });
65:     }
66: });
67:
68: app.post("/ai-review", async (req, res) => {
69:     const { code } = req.body;
70:     console.log("Code received for review:", code);
71:
72:     if (!code) {
73:         return res.status(400).json({ error: "Code is required" });
74:     }
75:
76:     try {
77:         const review = await aiCodeReview(code);
78:         console.log("AI Review output:", review);
79:         res.json({ review });
80:     } catch (error) {
81:         console.error("AI Review error:", error);
82:         res.status(500).json({ error: "AI Review failed: " + error.message });
83:     }
84: });
85:
86: const PORT = process.env.PORT || 7000;
87: app.listen(PORT, () => {
88:     console.log(Server is listening on port ${PORT}!);
89: });

Walkthrough
```

Lines 1-12: Imports Express and all compiler helper modules.

Lines 13-16: Enables cross-origin requests.

Lines 17-20: Defines a health endpoint.

Line 21: Defines the main code-execution endpoint.

Lines 22-23: Reads run request payload.

Lines 24-31: Rejects empty code submissions early.

Lines 32-33: Writes source code to disk.

Lines 34-37: Writes stdin data to disk.

Lines 38-17: Selects execution helper by language.

Lines 18-67: Returns output and temp file metadata.

Lines 68-76: Defines AI review endpoint.

Lines 77-85: Calls Gemini review helper.

Lines 86-89: Starts server on configured port.

### AI Review Helper

**File:** Compiler/aiCodeReview.js

**Purpose:** Wraps Google Gemini content generation to review submitted code.

**How it fits into service flow:** Called only by `/ai-review` in `Index.js`.

**Interview-ready talking points:**

- Loads Gemini API key from environment.
- Uses `gemini-2.5-flash` with a direct prompt that embeds the user's code.

**Implementation notes:**

- There is no prompt guard, usage limit, or output post-processing layer.

### Code

```
 1: const dotenv = require("dotenv");
 2: const { GoogleGenAI } = require("@google/genai");
 3:
 4: dotenv.config();
 5:
 6: const ai = new GoogleGenAI({ apiKey: process.env.GOOGLE_GEMINI_API_KEY });
 7:
 8: const aiCodeReview = async (code) => {
 9:   const response = await ai.models.generateContent({
10:     model: "gemini-2.5-flash",
11:     contents: Analyze and review the following code. Suggest improvements:\n\n${code},
12:   });
13:
14:   return response.text;
15: };
16:
17: module.exports = { aiCodeReview };

Walkthrough
```

Lines 1-3: Imports dotenv and Gemini SDK.

Lines 4-5: Loads Gemini API key from environment.

Lines 6-7: Creates Gemini client.

Line 8: Defines review helper.

Lines 9-13: Calls Gemini model.

Lines 14-17: Returns generated review text.

### Source File Generator

**File:** Compiler/generateFile.js

**Purpose:** Writes submitted code to a temp file in the `codes/` directory.

**How it fits into service flow:** Called before execution so native tools can compile or interpret a real file.

**Interview-ready talking points:**

- Uses UUIDs for most filenames.
- Java is special-cased to `Main.java` so class name matches executor expectations.

**Implementation notes:**

- Fixed `Main.java` naming can create collisions if multiple Java jobs run concurrently.

### Code

```
 1: const fs = require('fs');
 2: const path = require('path');
 3: const { v4: uuid } = require('uuid');
 4:
 5: const dirCodes = path.join(__dirname, 'codes');
 6:
 7: if (!fs.existsSync(dirCodes)) {
 8:     fs.mkdirSync(dirCodes, { recursive: true });
 9: }
10:
11: const generateFile = (format, content) => {
12:   const jobID = uuid();
13:   const filename = format === "java" ? "Main.java" : ${jobID}.${format};
14:   const filePath = path.join(dirCodes, filename);
15:   fs.writeFileSync(filePath, content);
16:   return filePath;
17: };
18:
19: module.exports = {
20:     generateFile,
21: };

Walkthrough
```

Lines 1-4: Imports filesystem, path, and UUID helpers.

Lines 5-6: Defines temp source directory.

Lines 7-10: Creates source directory if needed.

Line 11: Defines source-file generator.

Line 12: Creates unique job id.

Lines 13-14: Selects filename pattern.

Lines 15-21: Writes source code to disk.

### Input File Generator

**File:** Compiler/generateInputFile.js

**Purpose:** Writes stdin input to a temp `.txt` file in the `inputs/` directory.

**How it fits into service flow:** Called before execution so command-line programs can read redirected stdin.

**Interview-ready talking points:**

- Uses UUID-based filenames to isolate jobs.

### Code

```
 1: const fs = require('fs');
 2: const path = require('path');
 3: const { v4: uuid } = require('uuid');
 4:
 5: const dirInputs = path.join(__dirname, 'inputs');
 6:
 7: if (!fs.existsSync(dirInputs)) {
 8:     fs.mkdirSync(dirInputs, { recursive: true });
 9: }
10:
11: const generateInputFile = async (input) => {
12:     const jobID = uuid();
13:     const input_filename = ${jobID}.txt;
14:     const input_filePath = path.join(dirInputs, input_filename);
15:     await fs.writeFileSync(input_filePath, input);
16:     return input_filePath;
17: };
18:
19: module.exports = {
20:     generateInputFile,
21: };

Walkthrough
```

Lines 1-4: Imports filesystem, path, and UUID helpers.

Lines 5-6: Defines temp input directory.

Lines 7-10: Creates input directory if needed.

Line 11: Defines input-file generator.

Lines 12-14: Creates unique input file id.

Lines 15-21: Writes input text to disk.

### C Executor

**File:** Compiler/executeC.js

**Purpose:** Compiles C code with `gcc`, executes the compiled binary, and cleans temp files.

**How it fits into service flow:** Called by `/run` when language is `c`.

**Interview-ready talking points:**

- Run flow is compile command plus stdin redirection command combined with &&.

**Implementation notes:**

- Compilation and runtime errors are returned together via rejected object.

### Code

```
 1: const { exec } = require("child_process");
 2: const path = require("path");
```

```
 3: const { cleanupFiles } = require("./utils/cleanup");
 4:
 5: const executeC = (filepath, inputPath) => {
 6:   const dir = path.dirname(filepath);
 7:   const jobId = path.basename(filepath).split(".")[0];
 8:   const outPath = path.join(dir, ${jobId}.out);
 9:
10:   return new Promise((resolve, reject) => {
11:     const compileCmd = gcc "${filepath}" -o "${outPath}";
12:     const runCmd = "${outPath}" < "${inputPath}";
13:
14:     exec(${compileCmd} && ${runCmd}, (error, stdout, stderr) => {
15:       cleanupFiles([filepath, inputPath, outPath]);
16:
17:       if (error) {
18:         return reject({ error: error.message, stderr });
19:       }
20:
21:       resolve(stdout);
22:     });
23:   });
24: };
25:
26: module.exports = {
27:   executeC,
28: };

Walkthrough
```

Line 5: Defines C execution helper.

Lines 6-10: Computes output location.

Line 11: Builds compile command.

Lines 12-13: Builds run command with stdin redirection.

Line 14: Runs compile then execute.

Lines 15-28: Deletes temp files after execution.

### C++ Executor

**File:** Compiler/executeCpp.js

**Purpose:** Compiles C++ code with g++, executes the binary, and cleans temp files.

**How it fits into service flow:** Called by `/run` when language is `cpp`.

**Interview-ready talking points:**

- Compiled `.exe` is stored under `outputs/` before cleanup.

### Code

```
1: const { exec } = require("child_process");
2: const fs = require("fs");
3: const path = require("path");
4: const { cleanupFiles } = require("./utils/cleanup");
5:
6: const outputPath = path.join(__dirname, "outputs");
```

```
 7: if (!fs.existsSync(outputPath)) fs.mkdirSync(outputPath, { recursive: true });
 8:
 9: const executeCpp = (filepath, inputPath) => {
10:   const jobId = path.basename(filepath).split(".")[0];
11:   const outPath = path.join(outputPath, ${jobId}.exe);
12:
13:   return new Promise((resolve, reject) => {
14:     const compileCmd = g++ "${filepath}" -o "${outPath}";
15:     const runCmd = "${outPath}" < "${inputPath}";
16:
17:     exec(${compileCmd} && ${runCmd}, (error, stdout, stderr) => {
18:       cleanupFiles([filepath, inputPath, outPath]);
19:       if (error) return reject({ error: error.message, stderr });
20:       resolve(stdout);
21:     });
22:   });
23: };
24:
25: module.exports = { executeCpp };
26:

Walkthrough
```

Line 6: Defines compiled-binary output directory.

Lines 7-8: Creates binary output directory.

Lines 9-13: Defines C++ execution helper.

Lines 14-16: Builds compile command.

Line 17: Runs compile then execute.

Lines 18-26: Deletes temp files after completion.

### Go Executor

**File:** Compiler/executeGo.js

**Purpose:** Runs Go source directly with `go run` and cleans temp files.

**How it fits into service flow:** Called by `/run` when language is go.

**Interview-ready talking points:**

- Go path skips a separate compiled-output file and uses `go run` directly.

### Code

```
 1: const { exec } = require("child_process");
 2: const { cleanupFiles } = require("./utils/cleanup");
 3:
 4: const executeGo = (filepath, inputPath) => {
 5:   return new Promise((resolve, reject) => {
 6:     exec(go run "${filepath}" < "${inputPath}", (error, stdout, stderr) => {
 7:       cleanupFiles([filepath, inputPath]);
 8:       if (error) return reject({ error: error.message, stderr });
 9:       resolve(stdout);
10:     });
11:   });
12: };
```

```
13:
14: module.exports = { executeGo };

Walkthrough
```

Lines 4-5: Defines Go execution helper.

Line 6: Runs Go code with redirected stdin.

Lines 7-14: Deletes temp source and input files.

### Java Executor

**File:** Compiler/executeJava.js

**Purpose:** Compiles `Main.java`, runs the resulting class, and cleans generated files.

**How it fits into service flow:** Implemented but not currently wired into `/run` in `Index.js`.

**Interview-ready talking points:**

- Java compile/run flow is present and interview-worth noting even though it is currently unreachable from the route switch.

**Implementation notes:**

- If Java is re-enabled in UI and service switch, this executor is ready for that path.

### Code

```
 1: const { exec } = require("child_process");
 2: const path = require("path");
 3: const fs = require("fs");
 4: const { cleanupFiles } = require("./utils/cleanup");
 5:
 6: const executeJava = (filepath, inputPath) => {
 7:   const dir = path.dirname(filepath);
 8:   const className = "Main";
 9:
10:   const compileCmd = javac "${filepath}";
11:   const runCmd = java -cp "${dir}" ${className} < "${inputPath}";
12:
13:   return new Promise((resolve, reject) => {
14:     exec(${compileCmd} && ${runCmd}, (error, stdout, stderr) => {
15:       cleanupFiles([
16:         filepath,
17:         inputPath,
18:         path.join(dir, ${className}.class)
19:       ]);
20:
21:       if (error) return reject({ error: error.message, stderr });
22:       resolve(stdout);
23:     });
24:   });
25: };
26:
27: module.exports = { executeJava };

Walkthrough
```

Lines 6-7: Defines Java execution helper.

Lines 8-9: Uses fixed class name expected by generated file.

Line 10: Builds Java compile command.

Lines 11-13: Builds Java run command.

Line 14: Runs compile then execute.

Lines 15-27: Deletes `.java`, input, and compiled `.class` file.

### JavaScript Executor

**File:** Compiler/executeJs.js

**Purpose:** Runs JavaScript code with Node and cleans temp files.

**How it fits into service flow:** Called by `/run` when language is `javascript`.

**Interview-ready talking points:**

- Uses stdin redirection into node process.

### Code

```
 1: const { exec } = require("child_process");
 2: const { cleanupFiles } = require("./utils/cleanup");
 3:
 4: const executeJs = (filepath, inputPath) => {
 5:   return new Promise((resolve, reject) => {
 6:     exec(node "${filepath}" < "${inputPath}", (error, stdout, stderr) => {
 7:       cleanupFiles([filepath, inputPath]);
 8:       if (error) return reject({ error: error.message, stderr });
 9:       resolve(stdout);
10:     });
11:   });
12: };
13:
14: module.exports = { executeJs };

Walkthrough
```

Lines 4-5: Defines JavaScript execution helper.

Line 6: Runs Node against temp script file.

Lines 7-14: Deletes temp source and input files.

### Python Executor

**File:** Compiler/executePython.js

**Purpose:** Runs Python code with `python3` and cleans temp files.

**How it fits into service flow:** Called by `/run` when language is `python`.

**Interview-ready talking points:**

- Uses interpreter execution instead of compiling a binary first.

**Implementation notes:**

- The command assumes `python3` exists in PATH, which is true in the Docker image but can differ on Windows host setups.

### Code

```
 1: const { exec } = require("child_process");
 2: const { cleanupFiles } = require("./utils/cleanup");
 3:
 4: const executePython = (filepath, inputPath) => {
 5:   return new Promise((resolve, reject) => {
 6:     exec(python3 "${filepath}" < "${inputPath}", (error, stdout, stderr) => {
 7:       cleanupFiles([filepath, inputPath]);
 8:       if (error) return reject({ error: error.message, stderr });
 9:       resolve(stdout);
10:     });
11:   });
12: };
13:
14: module.exports = { executePython };

Walkthrough
```

Lines 4-5: Defines Python execution helper.

Line 6: Runs Python script with redirected stdin.

Lines 7-14: Deletes temp source and input files.

### Cleanup Utility

**File:** Compiler/utils/cleanup.js

**Purpose:** Deletes temp files asynchronously after execution completes.

**How it fits into service flow:** Called by every executor to remove generated source/input/output artifacts.

**Interview-ready talking points:**

- Keeps temp-file growth under control after each job.

**Implementation notes:**

- Cleanup is best-effort and logs failures except missing-file cases.

### Code

```
const fs = require("fs");

const cleanupFiles = (paths = []) => {
  for (const p of paths) {
    fs.unlink(p, (err) => {
      if (err && err.code !== 'ENOENT') {
        console.error(Failed to delete ${p}:, err.message);
      }
```

```
 9:     });
10:   }
11: };
12:
13: module.exports = { cleanupFiles };

Walkthrough
```

Line 3: Defines cleanup helper.

Line 4: Loops over temp file paths.

Line 5: Deletes each file asynchronously.

Lines 6-13: Logs real cleanup failures but ignores already-missing files.

## How To Explain This Compiler In Interviews

- Describe it as a separate execution microservice used by the main backend judge flow.
- Explain the file lifecycle: incoming code/input -> temp files -> native toolchain -> stdout -> cleanup.
- Mention that each language has its own executor wrapper around shell commands.
- Call out the risks: no sandboxing, no timeouts, no queueing, and environment/toolchain dependency.
- Mention AI review as a separate Gemini-backed endpoint, not part of correctness judging.